END-TO-END ENCRYPTED | AES-256-GCM | ZERO-KNOWLEDGE

StealthShare

End-to-End Encrypted File Sharing Platform

> "Upload. Encrypt. Share. Self-Destruct."

TypeScript	React 19	Express	PostgreSQL
Type-safe full-stack	Modern frontend	Node.js backend	Relational database

Built with AES-256-GCM Military-Grade Encryption

⚠ THE PROBLEM

The Problem With Traditional File Sharing

Google Drive

Server sees your files in **plaintext**. Your data is stored unencrypted and accessible to the provider at any time.

Email Attachments

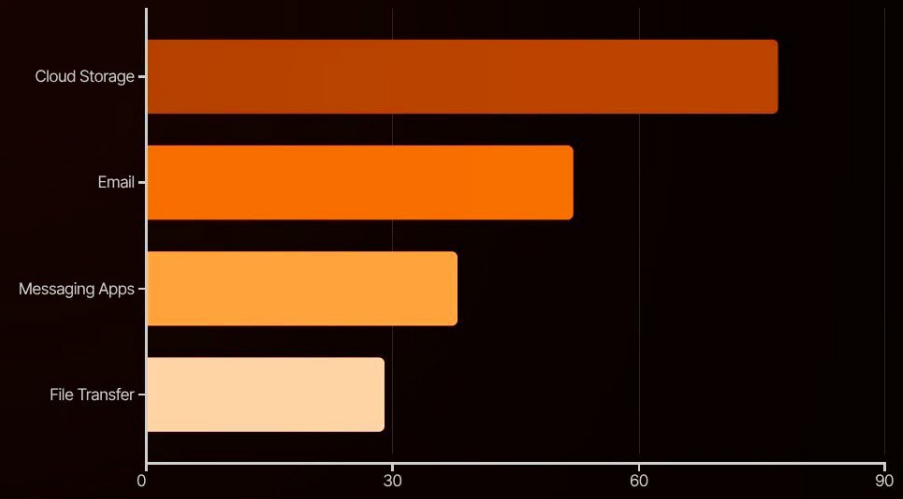
Files are **scanned by providers** for advertising and compliance purposes. Nothing is truly private.

WhatsApp / Telegram

Metadata is exposed — who you share with, when, and how often is all logged on central servers.

🔴 Key Vulnerabilities

- Files stored in plaintext on servers
- No automatic expiry or self-destruct
- Providers can access your files anytime
- No download limits = uncontrolled sharing
- Passwords transmitted to servers



Where Data Breaches Originate (%)

THE SOLUTION

StealthShare — The Solution

> "A zero-knowledge, end-to-end encrypted file sharing platform where your files are encrypted **BEFORE** they ever leave your browser."



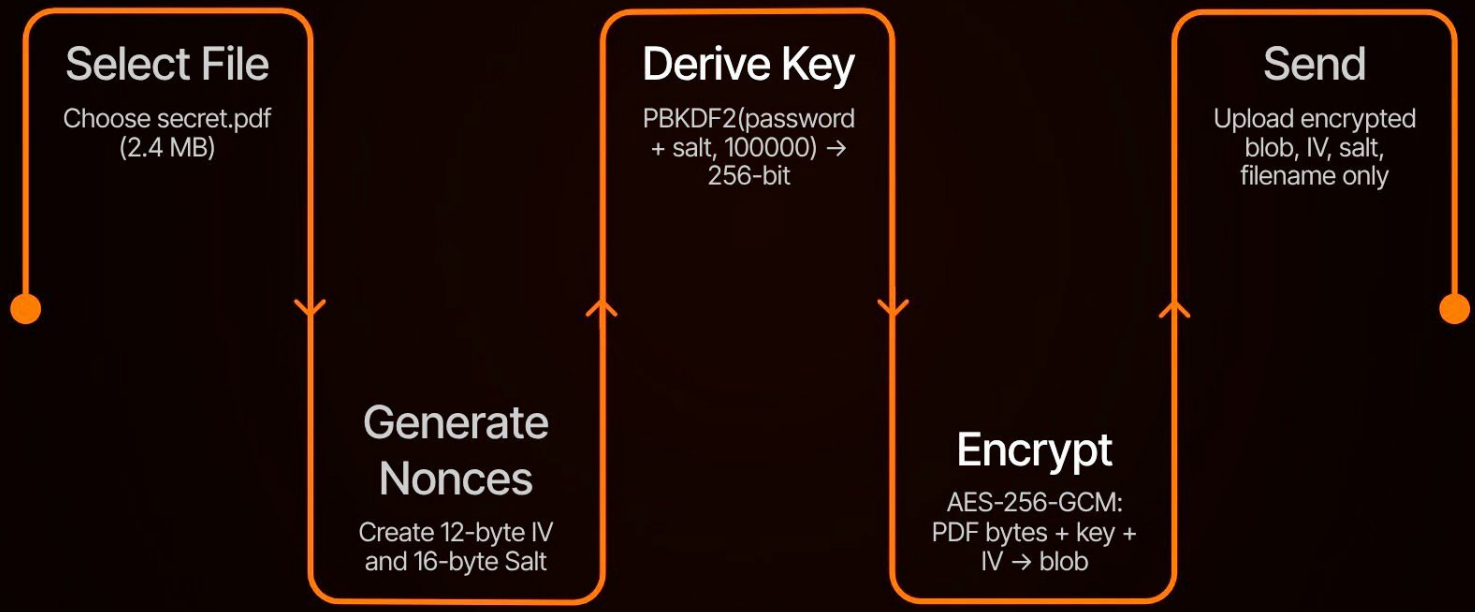
Every step of the process is designed to ensure your data never touches the server in readable form.

- Server NEVER sees your password
- Server NEVER sees your original file
- Files auto-delete after limits
- Military-grade AES-256-GCM

REAL EXAMPLE

What Happens When You Upload a PDF?

Everything happens inside YOUR browser



The cryptographic pipeline ensures that even if the server is compromised, attackers gain nothing useful — only meaningless ciphertext.

🔴 **Server receives: Encrypted blob only** — completely useless without the key. The password never leaves your browser under any circumstances.

ENCRYPTION DEEP DIVE

Military-Grade Encryption — How It Works

Web Crypto API running entirely in your browser

AES-256-GCM Explained

- **AES** = Advanced Encryption Standard
- **256** = 256-bit key (2^{256} combinations)
- **GCM** = Authenticated encryption mode

Used by: US Military, NSA, HTTPS, Signal, WhatsApp

Why 100,000 PBKDF2 Iterations?

Brute forcing 1 password:

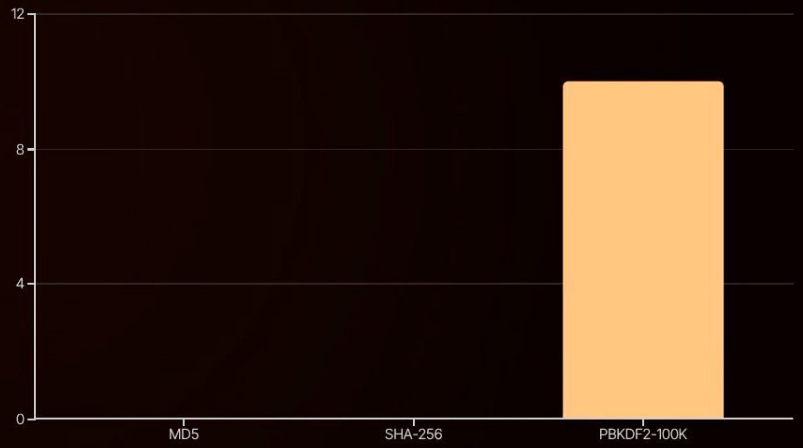
- Normal hash: **0.0001 seconds**
- PBKDF2 100K: **10 seconds**

Trying 1 million passwords:

- Normal: **1.7 minutes**
- PBKDF2: **115 DAYS**

Platform	Encrypted Where	Zero Knowledge
Google Drive	On Server	❌ No
WeTransfer	Not Encrypted	❌ No
StealthShare	**In Browser**	✅ Yes

Time to Brute Force Per Password



PBKDF2-100K (used by StealthShare) is **10,000× harder** to brute force than SHA-256 alone.

SHARE & SELF-DESTRUCT

Smart Sharing + Automatic Self-Destruct

You control every aspect of your shared file

⏰ Expiry Time

Set how many hours the link remains valid before it automatically expires

📥 Max Downloads

Set the maximum number of times the file can be downloaded

🔑 Password

Recipient must know your encryption password — never stored on server



Max Downloads?
Check download count

File Expired?
Check expiry time

Token Valid?
Authorize request token

Download Request
Incoming download attempt

The self-destruct system permanently removes files from disk and all database records — there is no recovery once triggered.

🔥 **Self-Destruct:** Delete from Disk + Delete Tokens from DB + Delete File Record — permanent and irreversible.

RECIPIENT JOURNEY

How the Recipient Gets Your PDF — Step by Step

Decryption happens in the recipient's browser, not the server

Two things you share:

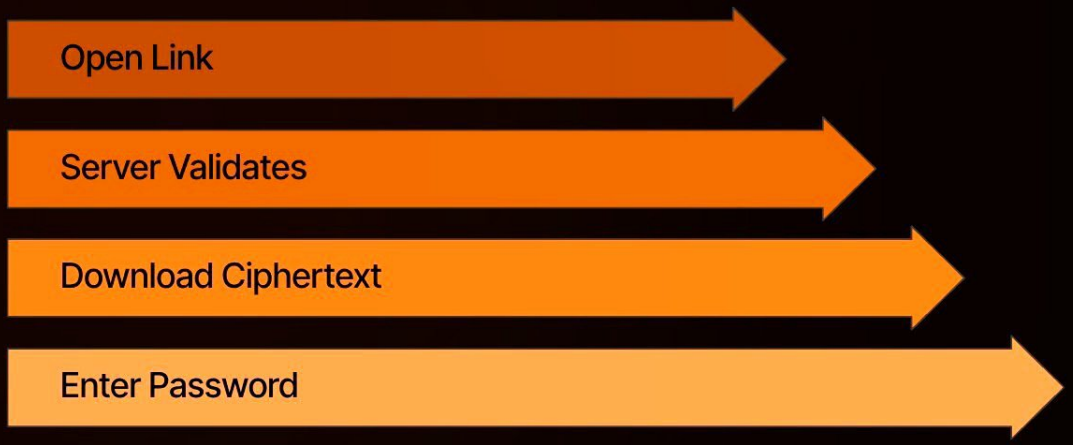
1️⃣ Share Link

stealthshare.app/download/a1b2c3...

2️⃣ Password

Sent separately via phone / in-person — never transmitted together with the link

⚠ ⚠️ **Wrong password?** Decryption fails **locally**. Server never knows the password was wrong!



TECH STACK

System Architecture — Monorepo Design

Three packages working together seamlessly

📱 CLIENT — React 19 + Vite + Tailwind CSS 4

Handles: UI, Encryption, Decryption, Web Crypto API

Port: localhost:5173 → /api/* proxy

🖥️ SERVER — Express 4 + TypeScript

Handles: Auth, File Storage, Token Generation, Self-Destruct

Port: localhost:3000 ↕️ Drizzle ORM

🗄️ PostgreSQL — Neon Serverless

Stores: Users, Encrypted Files, Access Tokens

📦 SHARED — Drizzle Schema + Zod Validation

Used by both client and server for type-safe data contracts

Frontend Stack

- React 19
- TypeScript
- Vite 6
- Tailwind 4
- Radix UI
- Web Crypto API

Backend Stack

- Express 4
- Drizzle ORM
- PostgreSQL
- Multer
- Zod
- Neon Serverless

SECURITY MODEL

Zero-Knowledge Security Model

The server is mathematically incapable of reading your files



Security Implementation Details

Feature	Implementation
Encryption	AES-256-GCM (authenticated)
Key Derivation	PBKDF2-SHA256, 100,000 iterations
IV	12 random bytes per file
Salt	16 random bytes per file
Auth Password	SHA-256 client-side hashed
Sessions	HttpOnly, SameSite=Lax, 24hr
Zero-Knowledge	Server sees only ciphertext
Self-Destruct	Disk + DB deletion on expiry



Zero-Knowledge

StealthShare score



Google Drive

Security score



WeTransfer

Security score

USE CASES

Who Needs StealthShare?

Perfect for anyone sharing sensitive information

💼 Business
Contracts, financial statements, legal documents

🏥 Medical
Patient records, prescriptions, medical reports

🔐 Personal
Passwords, private documents, personal media

🎓 Academic
Research data, thesis, confidential findings

💻 DevOps
SSH keys, .env files, API credentials

- 🔐 AES-256-GCM Encryption
- 🔑 PBKDF2 Key Derivation
- 🔗 Time-Limited Share Links
- 📦 Drag & Drop Upload
- 🛡️ Session Auth (SHA-256)

- ✅ Zero-Knowledge Server
- ✅ Self-Destruct Files
- ✅ Download-Capped Links
- ✅ Responsive Dark UI
- ✅ Full TypeScript Stack



StealthShare

Your files. Your keys. Your control.

Because Privacy Matters with StealthShare.

REACT 19 | EXPRESS | POSTGRESQL | WEB CRYPTO API